kNN-DBSCAN: algorithm, distributed implementation and correctness

A technical companion to Chen, Ruys and Biros, "KNN-DBSCAN: a DBSCAN in high dimensions"

knndbscan project
github.com/Prajwal-Prathiksh/knndbscan

November 2026 · knndbscan 0.2

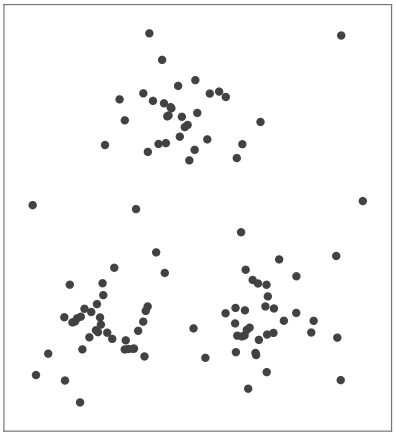
Abstract. kNN-DBSCAN replaces DBSCAN's range queries by a k-nearest-neighbor graph, which can be computed approximately and at scale in high dimensions. This companion explains the method of Chen, Ruys and Biros [1] to readers who want to understand, use or modify an implementation without first reading the paper. It follows one small example from start to end: why the graph classifies points exactly as DBSCAN does, why its clusters refine DBSCAN's, how Borůvka's algorithm grows them as trees, what changes on many machines, and which details an implementation must get right. Every idea is shown as a picture, explained in words and stated formally, and every claim about the implementation is checked by the project's tests.

Contents

1 What do you see? 1
2 Dense or sparse, one point at a time 3
3 Who belongs with whom 5
4 Clusters are trees 6
5 Many machines 11
6 Details that decide correctness 15
7 How we know it is right 17
8 Where the time goes 18
9 Using it 19
References 19

1 What do you see?

> *Look at these points. Which of them belong together?*

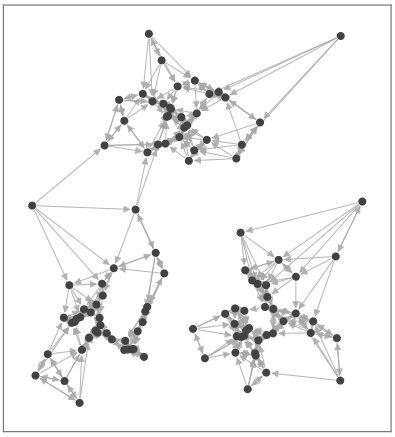


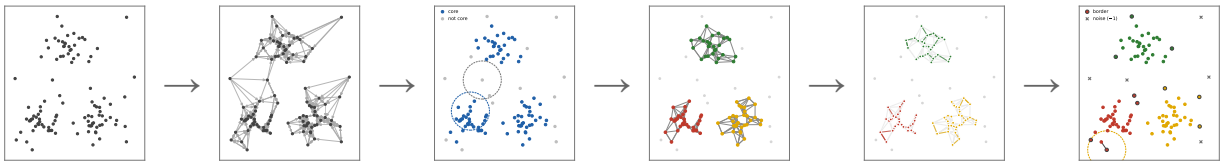
Most people see three groups and a handful of stragglers, without being told how many groups to look for or what shape they have. *Density-based clustering* turns that perception into a rule: a group is a region where points are packed closely, and a straggler is a point in a sparse region. DBSCAN [2] is the classic such rule. It needs no number of clusters, finds clusters of any shape, and sets outliers aside as noise.

Its cost is the question it asks of every point: *which points lie within distance ε of you?* In two dimensions a spatial tree answers this quickly. As the dimension grows, trees degrade towards comparing every pair of points, $O(n^2)$, and every new value of ε asks again.

kNN-DBSCAN [1] asks a different question: *who are your k nearest neighbors?* The answers form a k-nearest-neighbor graph (k-NNG): an arrow from each point to each of its k nearest neighbors. Unlike range queries, this graph can be approximated well in high dimensions (randomized projection trees [3], as in GOFMM [4]), it stores only $O(nk)$ edges, and it is computed once: every ε and every $M \leq k + 1$ can be clustered from it.

The surprise is how little is lost. The graph classifies every point exactly as DBSCAN does, and its clusters differ from DBSCAN's only by splitting a cluster that a thin bridge holds together.





points | kNN graph Section 1 | core points Section 2 | linked cores Section 3 | trees Section 4 | clusters, noise Section 2

Figure 1: The whole method on the running example, one frame per idea. Each later section explains one frame, then states it formally.

The running example has 107 points in three blobs, with $M = 6$ and $\varepsilon = 0.55$. Its panels are computed, not drawn: `figures/make_figures.py` replays the algorithm exactly as the pseudocode states it, stops unless the replay finds the library's clusters, and writes the numbers this text quotes.

1.1 The algorithm at a glance

All of kNN-DBSCAN fits in nine lines. They are given here once in full; whenever a later picture is about some of them, the algorithm appears again beside the picture with those lines highlighted, so there is never a reason to turn back.

Algorithm 1: kNN-DBSCAN on one rank: `knndbscan()` (paper's Algorithm 1). On a single process, lines 3–5 have nothing to exchange.

Require: this rank's rows of the kNN graph, sorted, without the point itself; ε; M
Ensure: a cluster label for each of this rank's points, -1 for noise

1 $C \leftarrow \{i :$ distance in column $M-1$ of row $i \leq \varepsilon\}$ — *// core points (Section 2)*
2 R, cut edges $\leftarrow$ local Borůvka over C — *// Algorithm 2*
3 number this rank's subtrees after those of lower ranks — *// `Allgather`*
4 rewrite each cut edge (i, j) as $(R[i], R[j])$ — *// `Alltoallv` to j's rank*
5 roots $\leftarrow$ cut MST over the subtrees — *// Algorithm 3*
6 $R[i] \leftarrow$ root of $R[i]$ for every $i \in C$
7 **parallel for** each non-core point i — *// border points*
8 $\quad q \leftarrow$ first core neighbor of i within ε, preferring one on this rank — *// `Alltoallv` if remote*
9 $\quad R[i] \leftarrow R[q]$ if q exists, otherwise -1 — *// noise*

1.2 How to read this companion

Each idea appears three times: as a **picture** of the running example, in plain **words**, and in **symbols**, as a definition, lemma or theorem in a tinted box. The pictures and the symbols say the same thing, so either can be used to check the other. Colours keep one meaning throughout: ● core point, ● border point, × noise, — an edge between processes, or what goes wrong; highlighted pseudocode lines are the ones the picture beside them shows. Numbers such as "Theorem 3.4" refer to the paper, whose statements are restated in one

notation; proofs are sketches that show why a result holds. `docs/ALGORITHM.md` maps each line of Algorithm 1 to its file and function.

In words	In symbols
the n points	$\mathcal{I}$
the distance between two points (Euclidean in the Python package)	$d(p, q)$
the radius, and the minimum number of points, the point itself included	ε, M (`eps`, `min_samples`)
the points within ε of p, p included	$\mathcal{N}_\varepsilon(p) = \{q \in \mathcal{I} : d(p, q) \leq \varepsilon\}$
the M points nearest to p, p included	$\mathcal{N}_M(p)$
the neighbors stored for each point, the point itself excluded	k
the core, border and noise points	$\mathcal{I}_{\text{core}}, \mathcal{I}_{\text{border}}, \mathcal{I}_{\text{noise}}$
the number of processes (MPI ranks)	P

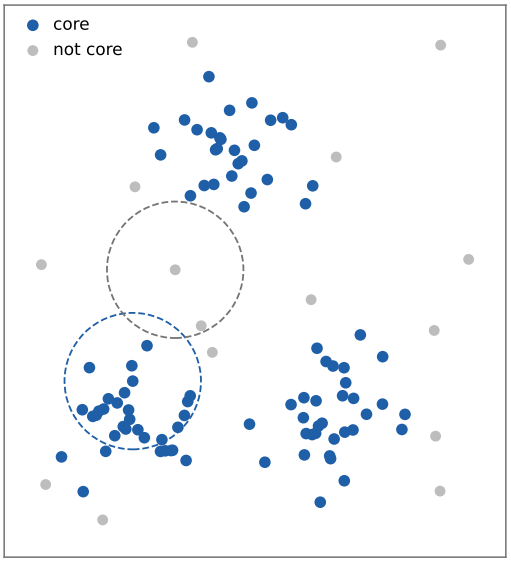
In one line. Cluster the kNN graph instead of ε-balls: the classification of points stays exactly DBSCAN's, and clustering becomes growing trees, which parallelizes.

2 Dense or sparse, one point at a time

> *When is a point in a dense place?*

Draw a disk of radius ε around the point and count the points inside, the point itself included. If there are at least M, the point is a **core point** ●: it lies inside a cluster. The blue dashed disk holds more than $M = 6$ points; the grey one holds fewer. Of the 107 points, 92 are core.

A point that is not core can still lie inside a core point's disk, at the edge of a cluster: it is a **border point**. A point in no core point's disk is **noise**.



Definition 1 (classification of points, Def. 3.1). Given ε and M, a point p is a *core point* if $|\mathcal{N}_\varepsilon(p)| \geq M$. A point that is not core is a *border point* if some core point q has $p \in \mathcal{N}_\varepsilon(q)$, and *noise* otherwise.

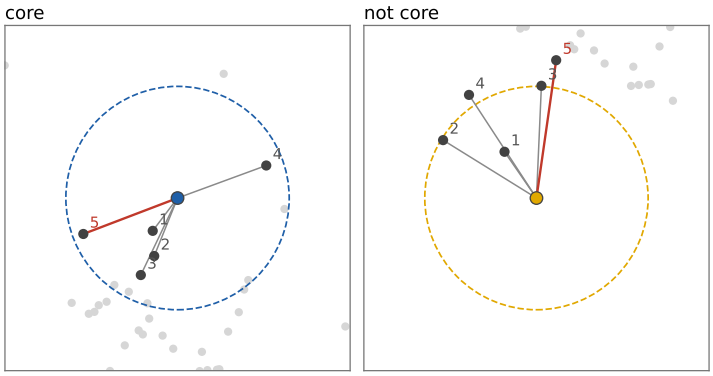
Counting points in a disk is a range query, the expensive operation. The kNN graph answers the same question without one.

1 Core points from one column

Algorithm 1

⋮
Ensure: a cluster label for each of this rank's points, -1 for noise
1 $C \leftarrow \{i$: distance in column $M-1$ of row $i \le \varepsilon\}$
2 R, cut edges $\leftarrow$ local Borůvka over C
⋮

Number a point's neighbors by distance, $1, 2, \ldots, M-1$. The disk holds M points, the point and $M-1$ others, exactly when neighbor $M-1$ (red) is inside it. Left, the 5th neighbor is inside: core. Right, it is just outside: not core. One number per point decides, the distance in column $M-1$ of its row, and no communication is needed.

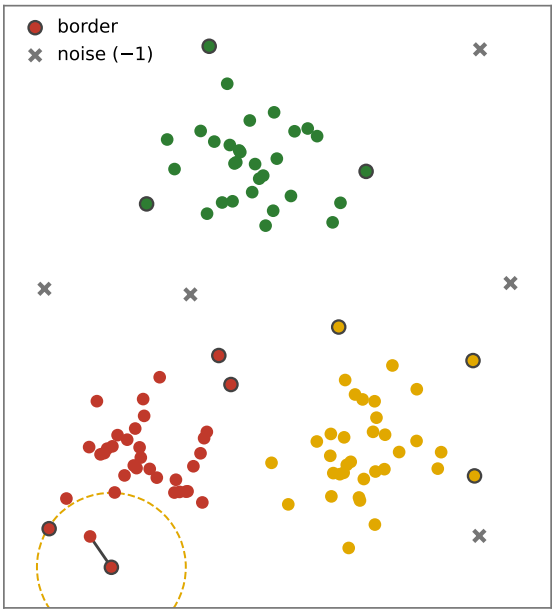


2 Border points and noise

Algorithm 1

⋮
6 $R[i] \leftarrow$ root of $R[i]$ for every $i \in C$
7 **parallel for** each non-core point i
8 $q \leftarrow$ first core neighbor of i within ε, preferring one on this rank
9 $R[i] \leftarrow R[q]$ if q exists, otherwise -1

A non-core point takes the cluster of its first core neighbor within ε (ringed; one is joined to the core point it follows) and is noise, -1, if it has none. The search never leaves the first $M-1$ columns: a core point that could make it a border point is always among its $M-1$ nearest neighbors. Here 10 points are border points and 5 are noise.



Lemma 1 (Lemma 3.1).
1. p is a core point if and only if $\max_{q \in \mathcal{N}_M(p)} d(p,q) \le \varepsilon$, that is, if its $(M-1)$-th nearest neighbor other than itself lies within ε.
2. p is a border point if and only if p is not core and some core point $q \in \mathcal{N}_M(p)$ has $d(p,q) \le \varepsilon$.

Proof. (1) $\mathcal{N}_\varepsilon(p)$ contains at least M points exactly when the M-th closest point (counting p) is within ε. (2) If p is not core, its M-th closest point is farther than ε, so every point within ε of p, in particular a core point that makes p a border point, is among its M nearest. □

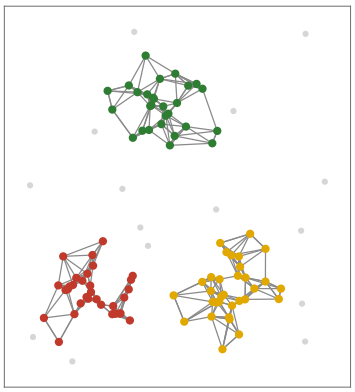
A k-NNG with $k \ge M-1$ therefore classifies every point exactly as DBSCAN does, whatever larger k was stored, and border points may join any cluster whose core point reaches them: that ambiguity is DBSCAN's own.

In one line. A point's class is read from its row: core if the distance in column $M-1$ is at most ε, border if a core point among its first $M-1$ neighbors is within ε.

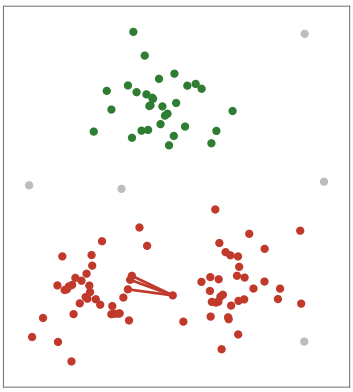
3 Who belongs with whom

> *Two core points are close to each other. Are they in the same cluster?*

DBSCAN says yes when they are within ε of each other, and links clusters through chains of such pairs. kNN-DBSCAN says yes only when one of them lists the other among its $M-1$ nearest neighbors. On the running example the two answers differ:

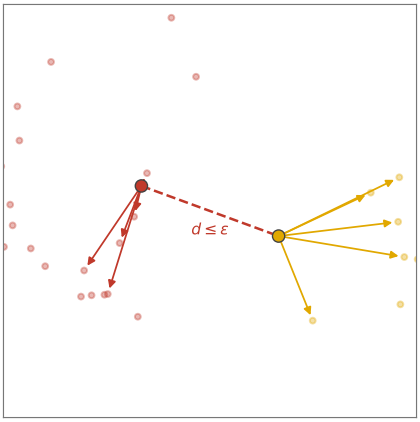


kNN-DBSCAN: 3 clusters



DBSCAN: 2 clusters

Zoom into the red link. Its two core points are within ε, so DBSCAN links them and merges two blobs. But each point's $M-1=5$ nearest neighbors (arrows) lie in its own blob, and the other point is not among them, so kNN-DBSCAN does not link them and keeps the blobs apart. Every kNN link is also a DBSCAN link (a core point's $M-1$ nearest neighbors are within ε), but not the other way round.



In words	In symbols
q is a step from core point p, for DBSCAN	$p \in \mathcal{I}_{\text{core}}$ and $d(p,q) \le \varepsilon$
q is a step from core point p, for kNN-DBSCAN: either one lists the other	$p \in \mathcal{I}_{\text{core}}$ and ($q \in \mathcal{N}_M(p)$ or $p \in \mathcal{N}_M(q)$)
core points in one kNN-DBSCAN cluster	connected in $G_{M,\,\text{core}}$: core points, joined to their core neighbors among the first $M-1$

Definition 2 (reachability and DBSCAN clusters, Defs. 3.2–3.5). q is *directly ε-reachable* from p if p is core and $q \in \mathcal{N}_\varepsilon(p)$; *$\varepsilon$-reachable* if a chain of core points $p = p_0, p_1, \ldots, p_m$ links them by direct reachability, ending in q. A *DBSCAN cluster* is a nonempty set C that is closed under ε-reachability (if $p \in C$ is core and q is reachable from p, then $q \in C$) and connected (any two members are reachable from a common core point).

Definition 3 (M-reachability and kNN-DBSCAN clusters, Defs. 3.6–3.9). q is *directly M-reachable* from p if p is core and $q \in \mathcal{N}_M(p)$ or $p \in \mathcal{N}_M(q)$. M-reachability, M-connectedness and *kNN-DBSCAN clusters* are defined from it exactly as their ε counterparts.

Lemma 2 (Lemmas 3.2 and 3.3). On core points, M-reachability is an equivalence relation, and it implies ε-reachability.

Proof. Symmetry comes from the symmetrized definition; reflexivity and transitivity from concatenating chains. If p is core then every $q \in \mathcal{N}_M(p)$ satisfies $d(p,q) \leq \varepsilon$ by Lemma 1, so each M-step is an ε-step. □

So the core points of each kNN-DBSCAN cluster are a connected component of $G_{M,\,\text{core}}$, and its edges need no distance test: they are within ε automatically.

Theorem 1 (relation to DBSCAN, Theorem 3.4). With the same ε and M:

1. every kNN-DBSCAN cluster's core points lie in a single DBSCAN cluster, so kNN-DBSCAN finds at least as many clusters as DBSCAN;
2. if the numbers of clusters are equal, the two clusterings of the core points coincide.

Core, border and noise points are the same in both (Lemma 1).

Proof. (1) follows from Lemma 2: M-connected core points are ε-connected. (2) By (1), sending each kNN-DBSCAN cluster to the DBSCAN cluster that contains its core points defines a map between the two sets of clusters. It is onto, because every core point belongs to some kNN-DBSCAN cluster. If the two sets have the same size, it is therefore one-to-one: no DBSCAN cluster holds two kNN-DBSCAN clusters, and the clusterings of the core points coincide. □

In the paper's experiments, and in this project's tests on blobs, moons, anisotropic and 20-dimensional data, the two clusterings agree on the core points whenever their counts agree, as part (2) guarantees.

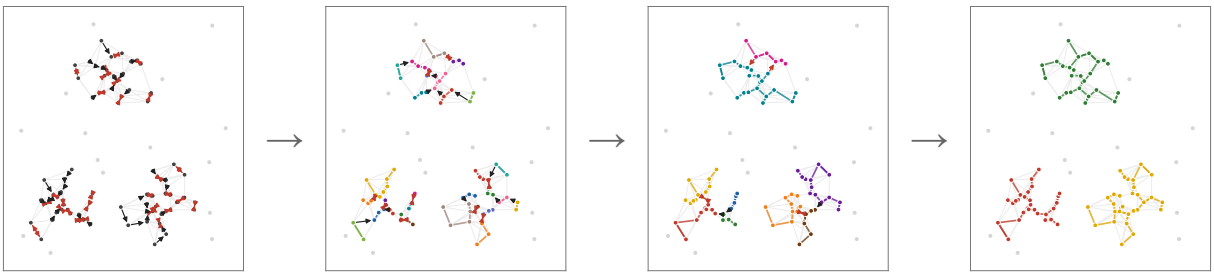
Consequences for the graph. Only the first $M-1$ columns of each row are ever read: they decide core points (column $M-1$), cluster edges (columns $1, \ldots, M-1$) and border candidates. A graph stored with k neighbors therefore serves every $M \leq k+1$ and every ε without recomputation, which makes parameter search cheap: the graph is the expensive part (Section 8).

In one line. kNN-DBSCAN links fewer pairs than DBSCAN, so its clusters refine DBSCAN's: it can split a cluster held together by a sparse bridge, and never merges two.

4 Clusters are trees

> *The clusters are the connected components of $G_{M,\,\text{core}}$. How are the components of a graph with billions of edges found in parallel?*

Grow a tree through each component. A spanning forest has exactly the components of its graph, and *Borůvka's algorithm* grows all of its trees at once: every tree, or *subtree*, picks its lightest edge to another subtree, and the picks merge subtrees. Every pick merges its subtree with another, so each round at least halves the number of subtrees that can still merge, and $O(\log n)$ rounds suffice.



round 1: 92 subtrees | round 2: 29 | round 3: 9 | 3 trees: the clusters

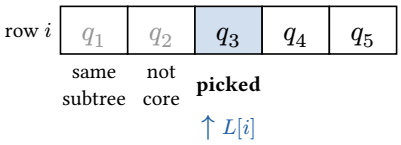
Figure 2: Borůvka on the core points of the running example. Arrows are the picks of a round, colours the subtrees it starts from, red arrows pairs of subtrees that picked each other. Each round's picks merge the subtrees into the next frame's.

Theorem 2 (Theorem 3.5). The connected components of a minimum spanning forest of $G_{M,\,\text{core}}$ are the kNN-DBSCAN clusters of the core points (and those of $G_{\varepsilon,\,\text{core}}$ are the DBSCAN clusters).

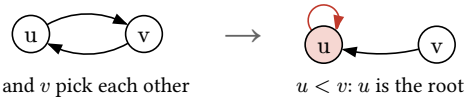
The statement is immediate, since a spanning forest has its graph's components; its value is algorithmic. A *minimum* spanning forest also supports hierarchical clustering; for the clusters alone minimality is not needed (Proposition 1). A round has three mechanics.

4.1 Picking: the lightest edge is the first eligible one

Each row lists a point's neighbors in increasing distance, so the first neighbor that is a core point in another subtree is the point's lightest such edge, and the scan stops there. Each subtree keeps the lightest edge its points propose. An edge that has become internal to a subtree stays internal, so the column pointer $L[i]$ only moves forward: over all rounds, each of the $n(M-1)$ edges is inspected a bounded number of times.



4.2 Breaking symmetry: a mutual pair elects a root



Each subtree u points to the subtree $p(u)$ its edge reaches, or to itself if it has no edge. Two subtrees that picked each other would point at each other forever, so the one with the smaller id points at itself and becomes the root of both. In the running example every new subtree contains exactly one such pair: 29 pairs in round 1 for 29 new subtrees, 9 in round 2 for 9.

4.3 Pointer jumping: everyone finds the root in log steps

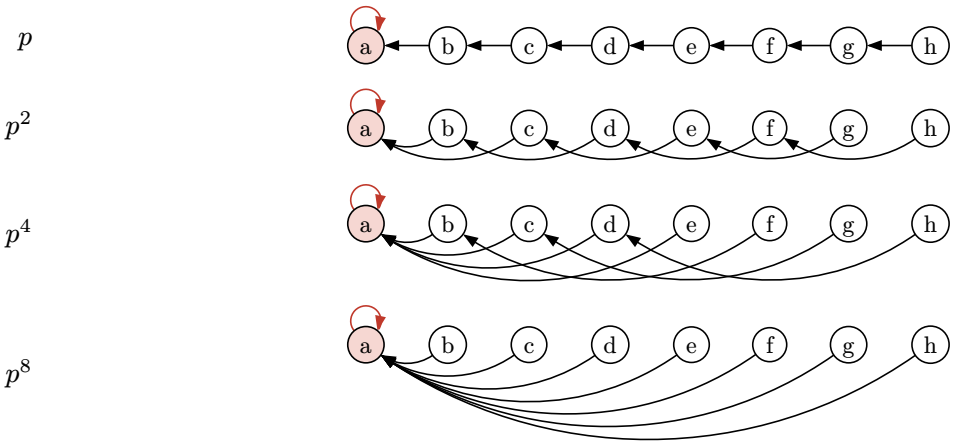


Figure 3: Pointer jumping on a chain of depth 7: every node replaces its pointer by its pointer's pointer, $p \leftarrow p \circ p$, so after t rounds it points 2^t steps ahead, or at the root. After $\lceil \log_2 7 \rceil = 3$ rounds every node points at the root a.

After pointer jumping every subtree points at the root of its new tree, and each core point takes that root as its label. With the three mechanics, the whole local algorithm:

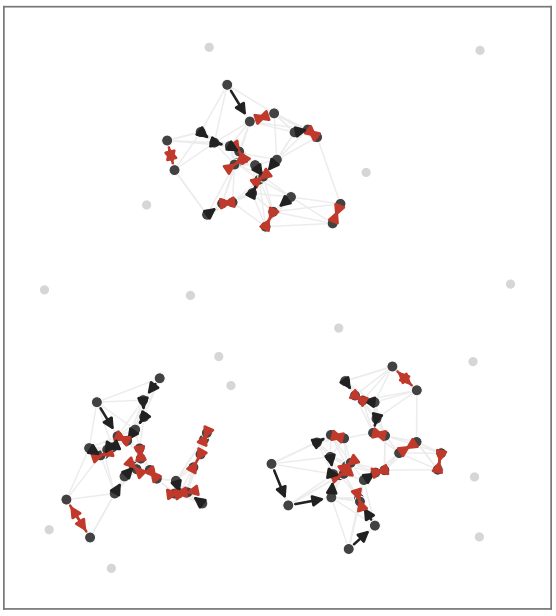
Algorithm 2: Local Borůvka over the core points of one process (paper's Algorithm 2). `pwrite` keeps the lighter of two proposals (paper's Algorithm 3).

Require: core points C of this process; neighbor rows sorted by distance
Ensure: $R[i]$, the root of the subtree that contains each $i \in C$
$R[i] \leftarrow i$ and $L[i] \leftarrow 1$ for every $i \in C$ *// $L[i]$: next column of row i*
repeat
  $E_{\min}[u] \leftarrow (\text{none}, +\infty)$ for every subtree u
  parallel for $i \in C$ *// pick*
    while $L[i] \leq M - 1$
      $q \leftarrow$ neighbor $L[i]$ of i
      if q is stored on another rank **then** record cut edge (i, q); $L[i] \leftarrow L[i] + 1$
      else if $q \in C$ **and** $R[q] \neq R[i]$ **then** `pwrite`$(E_{\min}[R[i]], R[q], d(i, q))$; **break** *// rows are sorted*
      else $L[i] \leftarrow L[i] + 1$ *// non-core, or same subtree*
  $p(u) \leftarrow E_{\min}[u]$.target, or u if it has none *// break symmetry*
  if $p(p(u)) = u$ **and** $u < p(u)$ **then** $p(u) \leftarrow u$ *// mutual pair: smaller id is root*
  repeat $p(u) \leftarrow p(p(u))$ **until** no pointer changes *// pointer jumping*
  join the subtrees left on cycles *// Section 4.4*
  $R[i] \leftarrow p(R[i])$ for every $i \in C$ *// relabel*
until the number of subtrees does not decrease

1 Round 1: every subtree picks

Algorithm 2

Require: core points C of this process; neighbor rows sorted by distance
Ensure: $R[i]$, the root of the subtree that contains each $i \in C$
$R[i] \leftarrow i$ and $L[i] \leftarrow 1$ for every $i \in C$
repeat
  $E_{\min}[u] \leftarrow (\text{none}, +\infty)$ for every subtree u
  parallel for $i \in C$
    while $L[i] \leq M - 1$
      $q \leftarrow$ neighbor $L[i]$ of i
      if q is stored on another rank **then** record cut edge (i, q); $L[i] \leftarrow L[i] + 1$
      else if $q \in C$ **and** $R[q] \neq R[i]$ **then** `pwrite` $(E_{\min}[R[i]], R[q], d(i, q))$; **break**
      else $L[i] \leftarrow L[i] + 1$
  $p(u) \leftarrow E_{\min}[u]$.target, or u if it has none
  if $p(p(u)) = u$ **and** $u < p(u)$ **then** $p(u) \leftarrow u$
  repeat $p(u) \leftarrow p(p(u))$ **until** no pointer changes
  join the subtrees left on cycles
  $R[i] \leftarrow p(R[i])$ for every $i \in C$
until the number of subtrees does not decrease

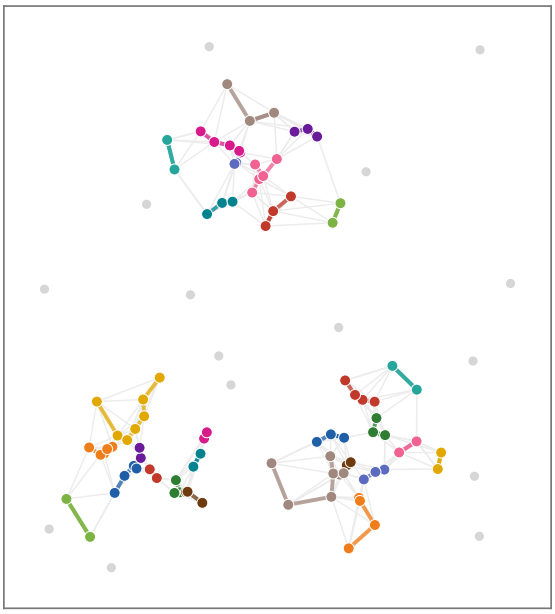


Every core point starts as a subtree of its own (dark). It scans its row and stops at the first core neighbor in another subtree, its lightest edge (arrows). Red arrows are the 29 pairs that picked each other.

2 Round 1: merge along the picks

Algorithm 2

Require: core points C of this process; neighbor rows sorted by distance
Ensure: $R[i]$, the root of the subtree that contains each $i \in C$
1 $R[i] \leftarrow i$ and $L[i] \leftarrow 1$ for every $i \in C$
2 **repeat**
3 $E_{\min}[u] \leftarrow (\text{none}, +\infty)$ for every subtree u
4 **parallel for** $i \in C$
5 **while** $L[i] \leq M - 1$
6 $q \leftarrow$ neighbor $L[i]$ of i
7 **if** q is stored on another rank **then** record cut edge (i, q); $L[i] \leftarrow L[i] + 1$
8 **else if** $q \in C$ **and** $R[q] \neq R[i]$ **then** `pwrite` $(E_{\min}[R[i]], R[q], d(i, q))$; **break**
9 **else** $L[i] \leftarrow L[i] + 1$
10 $p(u) \leftarrow E_{\min}[u]$.target, or u if it has none
11 **if** $p(p(u)) = u$ **and** $u < p(u)$ **then** $p(u) \leftarrow u$
12 **repeat** $p(u) \leftarrow p(p(u))$ **until** no pointer changes
13 join the subtrees left on cycles
14 $R[i] \leftarrow p(R[i])$ for every $i \in C$
15 **until** the number of subtrees does not decrease

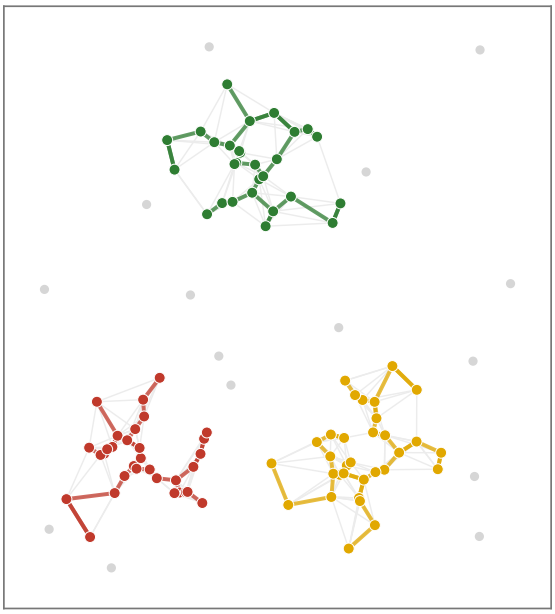


Each subtree points along its pick; each mutual pair elects its smaller id; pointer jumping takes every subtree to its root, and relabelling hands the root to every point. The 92 subtrees become 29, drawn with the edges of their trees.

3 No edge left: the clusters

Algorithm 2

Require: core points C of this process; neighbor rows sorted by distance
Ensure: $R[i]$, the root of the subtree that contains each $i \in C$
1 $R[i] \leftarrow i$ and $L[i] \leftarrow 1$ for every $i \in C$
2 **repeat**
3 $E_{\min}[u] \leftarrow (\text{none}, +\infty)$ for every subtree u
4 **parallel for** $i \in C$
5 **while** $L[i] \leq M - 1$
6 $q \leftarrow$ neighbor $L[i]$ of i
7 **if** q is stored on another rank **then** record cut edge (i, q); $L[i] \leftarrow L[i] + 1$
8 **else if** $q \in C$ **and** $R[q] \neq R[i]$ **then** `pwrite` $(E_{\min}[R[i]], R[q], d(i, q))$; **break**
9 **else** $L[i] \leftarrow L[i] + 1$
10 $p(u) \leftarrow E_{\min}[u]$.target, or u if it has none
11 **if** $p(p(u)) = u$ **and** $u < p(u)$ **then** $p(u) \leftarrow u$
12 **repeat** $p(u) \leftarrow p(p(u))$ **until** no pointer changes
13 join the subtrees left on cycles
14 $R[i] \leftarrow p(R[i])$ for every $i \in C$
15 **until** the number of subtrees does not decrease



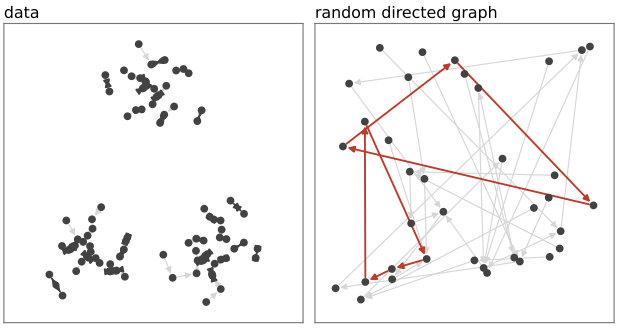
Rounds 2 and 3 repeat lines 3–14 (Figure 2). Then no subtree has an edge to another, the number of subtrees stays at 3, and the loop ends. The coloured edges are a spanning forest of $G_{M,\,\text{core}}$, one tree per cluster.

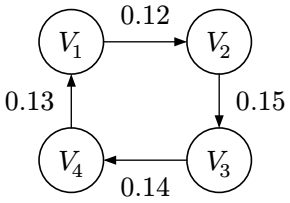
4.4 When nobody is a root

> *Symmetry breaking assumes the picks end in a mutual pair. What if they never do?*

On data they always do. With an undirected graph and distinct weights, following picks only ever decreases the weight, so a chain of picks can only end by returning to the previous subtree: a mutual pair. The example's first round has 29 pairs and 0 longer cycles (left).

A kNN graph is *directed*, though: $q \in \mathcal{N}_M(p)$ does not imply $p \in \mathcal{N}_M(q)$, so a subtree sees only the edges its own points list, and the weights along a chain of picks need not decrease. On a random directed graph (right) the picks form no pairs at all, but cycles of length 3 and 4 (red). Ties, frequent in duplicated or quantized data, create such cycles in real data too.





The paper's smallest example ($k = 2$): each vertex's lightest out-edge is drawn. $V_1 \to V_2$ is the lightest edge of all, so an undirected graph would make V_1, V_2 a mutual pair. But V_2 does not list V_1, and the picks close a cycle of four. None of its members points at itself, so none is a root, and pointer jumping has to cope with it.

What pointer jumping does on a cycle of length ℓ depends on ℓ. After t rounds each pointer is p^{2^t}, which is the identity exactly when ℓ divides 2^t:

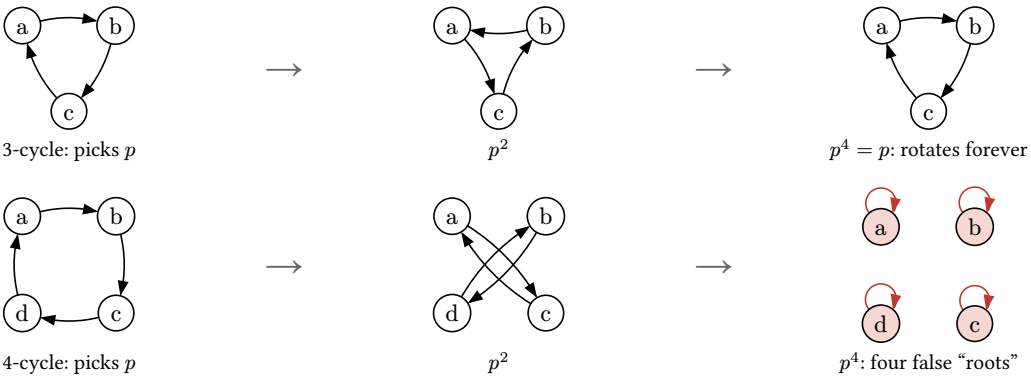


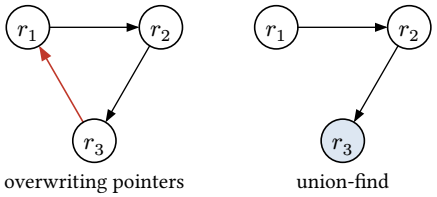
Figure 4: Pointer jumping on cycles. On an odd cycle (top) the pointers rotate and the number of unresolved subtrees stops falling, which is easy to detect. On a cycle whose length is a power of two (bottom) every member ends pointing at itself, which is exactly what a root looks like (shaded).

Power-of-two cycles. An implementation that detects cycles only by the count of unresolved subtrees catches odd cycles, but on a cycle of length 2, 4, 8, … every member "resolves" to itself. Taken as roots, the members never merge, the next round picks the same edges, and Borůvka stops with the cluster in pieces. The fix: a subtree that was not a root and whose chain returns to itself is on a cycle, and stays unresolved so that cycle joining sees it. Sequential in-place updates hide the problem, because the first member to point at itself becomes the others' root; the synchronous updates of the distributed algorithm expose it (Section 5).

The subtrees left unresolved form cycles with trees hanging off them. Line 13 of Algorithm 2 joins each such group by making one member the root of all of it, and both ways it can go wrong produce a wrong cluster:

1 Grouping must link roots

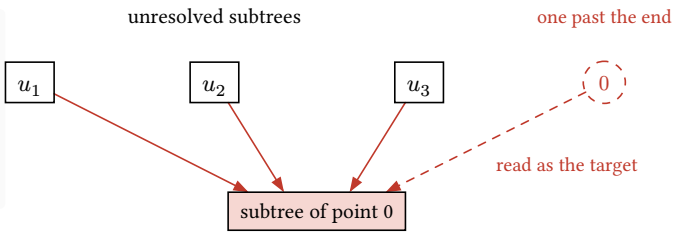
Grouping joins the members along their pointers. Overwriting one representative's pointer with another's, without first following each to its root, can close a loop among representatives (red), and the pointer jumping that follows never ends: a run on 5 ranks hung this way. A union-find links only roots, so each group stays a tree with one root (shaded).



2 Reading past the list

Algorithm 2

```
    ⋮
11  if p(p(u)) = u and u < p(u) then p(u) ← u
12  repeat p(u) ← p(p(u)) until no pointer changes
13  join the subtrees left on cycles
14  R[i] ← p(R[i]) for every i ∈ C
15  until the number of subtrees does not decrease
```



The shared-memory grouping read one element past its list of unresolved subtrees. The stray value was usually 0, so it formed no groups and pointed every unresolved subtree at the subtree of point 0, merging them into whatever cluster point 0 was in. AddressSanitizer reported the read. Only multithreaded runs reach this line: sequential in-place updates collapse cycles on their own.

4.5 Picks need not be lightest

The paper's minimum-edge update (`pwrite`) is a compare-and-swap loop. If the compare and the write are not one atomic operation, two threads proposing to the same subtree can leave an edge that is not the lightest, or the target of one proposal with the weight of another. Neither changes the clusters:

> **Proposition 1** (Borůvka with arbitrary outgoing edges). Suppose each round, every subtree with at least one outgoing edge records *some* outgoing edge (not necessarily the lightest), and subtrees are merged along the recorded edges with cycles joined as above. Then the procedure terminates, and its final subtrees are exactly the connected components of the graph.

Proof. Safety: every merge follows an edge of the graph, so no subtree ever spans two components. *Progress:* while some component contains two subtrees, each of them has an outgoing edge and records one, and following recorded edges from any of them leads into a merge, so the number of subtrees strictly decreases. The procedure stops only when no subtree has an outgoing edge, that is, when each subtree is a whole component. □

Minimality matters for the tree, and for hierarchical clustering, not for the clusters. The proposition also explains why the clusters do not depend on thread scheduling though the trees, and the order in which roots are found, do; labels are therefore renumbered at the end (Section 6).

> **In one line.** Borůvka finds the components in $O(\log n)$ rounds of "pick an edge, merge along the picks", provided the cycles of picks that a directed graph creates are detected and joined.

5 Many machines

> *The graph does not fit on one machine. What has to change?*

With P ranks the points are split into contiguous blocks in rank order: with $n_0 = \lceil n/P \rceil$, rank r holds points $rn_0, \ldots, (r+1)n_0 - 1$, the last nonempty rank holds the remainder, and trailing ranks may hold nothing (7 points on 6 ranks leave ranks 4 and 5 empty). Each rank stores only its own rows, with global neighbor ids.

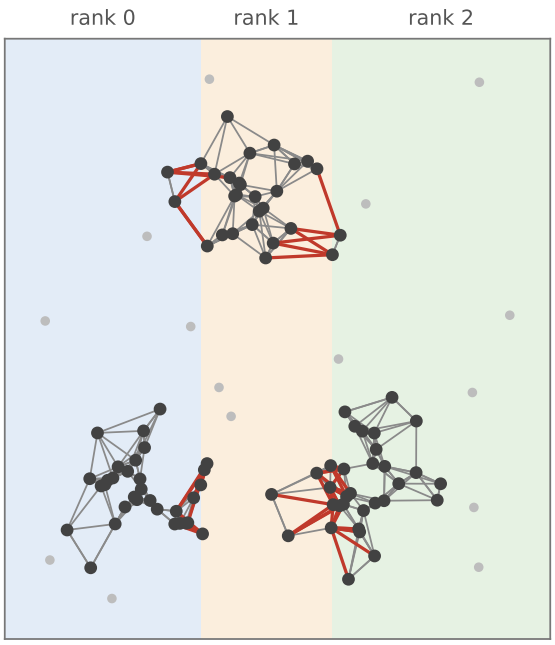
The plan: grow trees inside each rank without talking, then treat each tree as a single vertex and run Borůvka once more across ranks, on a graph far smaller than the original.

1 Blocks and cut edges

Algorithm 1

Require: this rank's rows of the kNN graph, sorted, without the point itself; ε; M
Ensure: a cluster label for each of this rank's points, -1 for noise
1 $C \leftarrow \{i : \text{distance in column } M-1 \text{ of row } i \le \varepsilon\}$
2 R, cut edges $\leftarrow$ local Borůvka over C
3 number this rank's subtrees after those of lower ranks
⋮

The running example on 3 ranks, with points numbered left to right so that blocks are strips. Each rank finds its core points alone. An edge to a point of another rank is a *cut edge* —: there are 61.

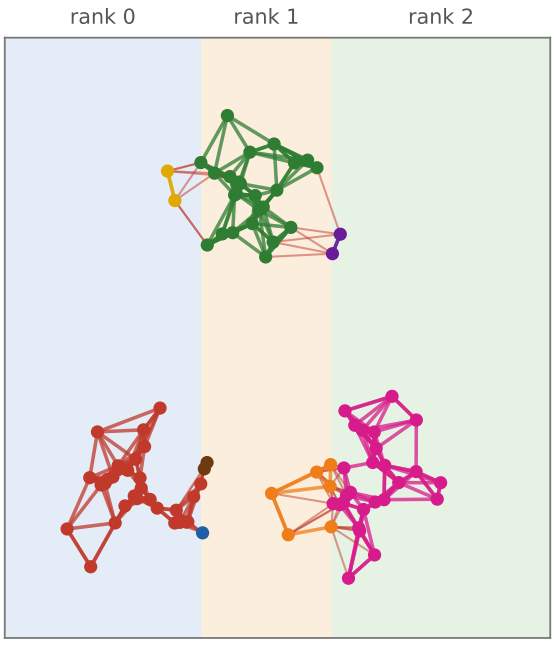


2 Local Borůvka, without communication

Algorithm 1

⋮
Ensure: a cluster label for each of this rank's points, -1 for noise
1 $C \leftarrow \{i : \text{distance in column } M-1 \text{ of row } i \le \varepsilon\}$
2 R, cut edges $\leftarrow$ local Borůvka over C
3 number this rank's subtrees after those of lower ranks
4 rewrite each cut edge (i, j) as $(R[i], R[j])$
⋮

Each rank runs Algorithm 2 on its block, recording cut edges instead of following them. This gives 8 subtrees, some of a single point: a cluster that spans ranks is still in pieces.

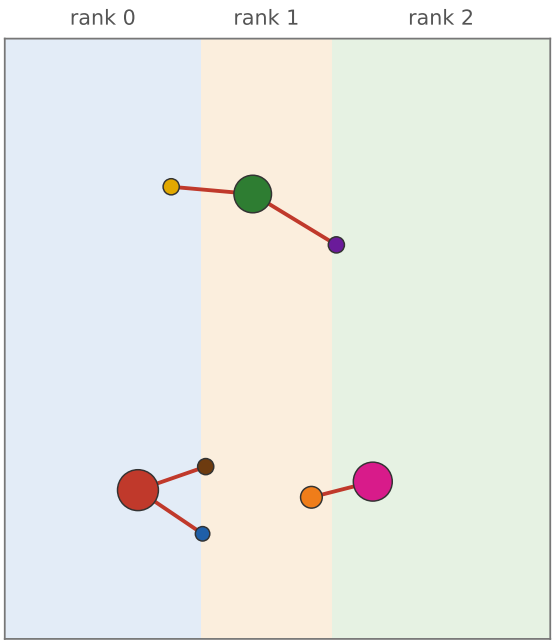


3 Subtrees become vertices

Algorithm 1

```
    ⋮
1   C ← {i : distance in column M − 1 of row i ≤ ε}
2   R, cut edges ← local Borůvka over C
3   number this rank's subtrees after those of lower ranks
4   rewrite each cut edge (i, j) as (R[i], R[j])
5   roots ← cut MST over the subtrees
6   R[i] ← root of R[i] for every i ∈ C
    ⋮
```

Each subtree gets a global id, numbered after those of lower ranks, and the rank that numbered it *owns* it. Each cut edge between points becomes an edge between their subtrees, which only the far point's rank can name: 5 links here.

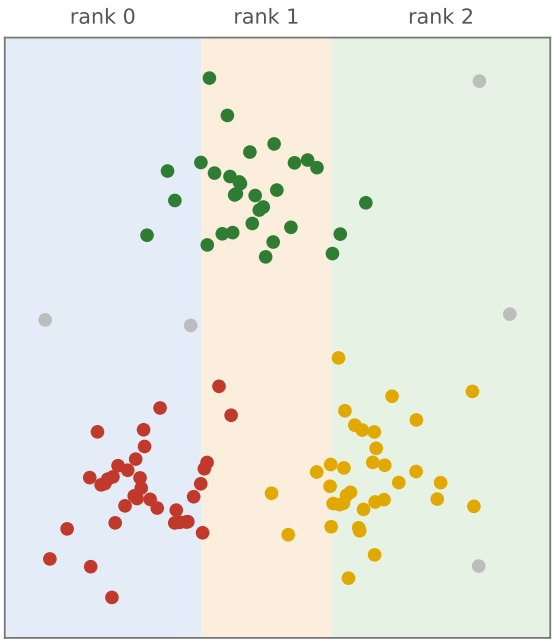


4 Cut MST: the same clusters

Algorithm 1

```
    ⋮
3   number this rank's subtrees after those of lower ranks
4   rewrite each cut edge (i, j) as (R[i], R[j])
5   roots ← cut MST over the subtrees
6   R[i] ← root of R[i] for every i ∈ C
7   parallel for each non-core point i
8       q ← first core neighbor of i within ε, preferring one
        on this rank
    ⋮
```

Borůvka once more, over the subtrees and across ranks (Algorithm 3), and every point takes its subtree's root. The clusters are those found on one rank, as the next theorem guarantees for any split.



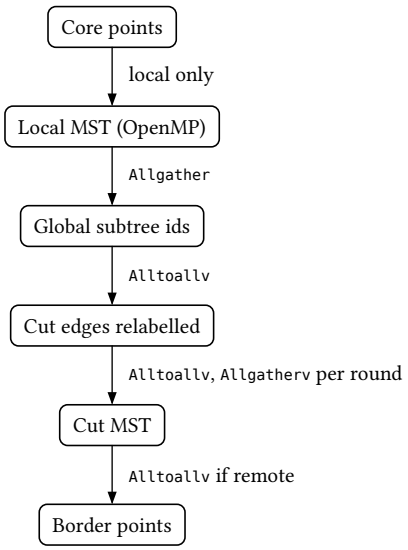
Theorem 3 (inexact MST, Theorem 3.6). Partition the vertices into groups. Build a minimum spanning forest of each group using only edges inside the group; contract each resulting tree to a super-vertex; build a spanning forest of the super-vertices using only the cut edges between groups. The union of these forests is generally not a minimum spanning forest of the whole graph, but its connected components are those of the graph.

Proof. Any path between two vertices alternates runs of internal edges, whose endpoints the group forests already connect, with cut edges, which the forest over super-vertices connects. Conversely every forest edge is a graph edge. □

The theorem is also the property an implementation must keep: **the clusters of core points do not depend on the number of ranks or on how points are assigned to them.** Border points are the exception, by DBSCAN's own ambiguity: one with core neighbors in several clusters prefers a neighbor on its own rank, so its cluster may change with the rank count.

Each rank runs the stages of Algorithm 1 in order, and only these transitions communicate. The communication is collective throughout: `Allgather` and `Allgatherv` for counts and roots, `Alltoallv` for edge proposals and pointer queries, and `Gatherv` and `Scatterv` for the rare cycle joining on rank 0.

The paper's analysis gives a local-stage work of $O(\frac{n}{P} \cdot M + (\frac{n}{P}) \log(\frac{n}{P}))$ per rank, and cut-stage communication that grows with the number of cut edges per rank and with $\log P$ rounds. Cut edges increase with the dimension of the data, which is what limits strong scaling in high dimensions.



5.1 Borůvka across ranks

The cut MST is Algorithm 2 with each step spread over the ranks: the owner of a subtree collects its proposals, and pointer jumping asks the owner of $p(u)$ for $p(p(u))$. Every rank updates its pointers at once, which is where the power-of-two cycles of Figure 4 appear.

Algorithm 3: Cut MST: Borůvka over subtrees across ranks (paper's Algorithm 4).

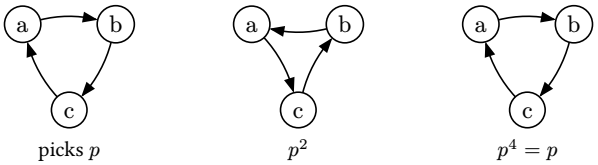
Require: subtrees, each owned by one rank; cut edges (u, v, w) between subtree ids
Ensure: the root of every subtree

repeat
  parallel for each subtree u this rank touches: its lightest local cut edge — *// local proposal*
  send each proposal to the owner of u, which keeps the lightest as $E_{\min}[u]$ — *// `Alltoallv`*
  owners break symmetry between mutual pairs — *// as in Algorithm 2*
  repeat — *// distributed pointer jumping*
    for each unresolved u, ask the owner of $p(u)$ for $p(p(u))$ — *// `Alltoallv` request and reply*
    keep u unresolved if its chain returns to u — *// power-of-two cycles*
  until every subtree is resolved, **or** the unresolved count stalls
  if stalled **then** group the unresolved subtrees on rank 0 with a union-find — *// `Gatherv`, `Scatterv`*
  share the new roots; relabel cut edges, dropping those inside one subtree — *// `Allgatherv`*
until the global number of subtrees does not decrease

1 An odd cycle stalls

Algorithm 3

```
    ⋮
4   owners break symmetry between mutual pairs
5   repeat
6       for each unresolved u, ask the owner of p(u) for p(p(u))
7       keep u unresolved if its chain returns to u
8   until every subtree is resolved, or the unresolved count stalls
9   if stalled then group the unresolved subtrees on rank 0 with a union-find
10  share the new roots; relabel cut edges, dropping those inside one subtree
    ⋮
```

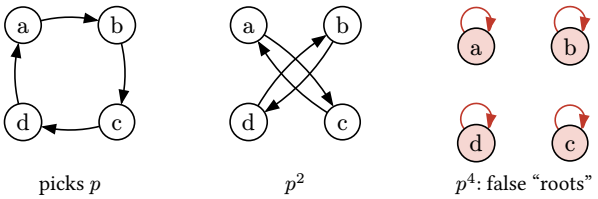


Three subtrees, owned by any ranks, pick each other in a ring. Every round asks the owners for the pointers' pointers, which only rotate: the count of unresolved subtrees stops falling, the stall is detected, and rank 0 groups the ring with a union-find.

2 A power-of-two cycle must stay unresolved

Algorithm 3

```
    ⋮
4   owners break symmetry between mutual pairs
5   repeat
6       for each unresolved u, ask the owner of p(u) for p(p(u))
7       keep u unresolved if its chain returns to u
8   until every subtree is resolved, or the unresolved count stalls
    ⋮
```



Four subtrees in a ring point at themselves after two rounds and would pass for four roots; the cluster would stay in four pieces on some rank counts (one cluster became six on 7 ranks). Line 7 keeps a subtree whose chain returned to itself unresolved, so the ring stalls like an odd one and is grouped.

The Python package runs the same code on a single rank, through a stand-in that implements each collective for a one-rank communicator, so stages III–V become bookkeeping. The MPI command-line program runs it on any number of ranks.

In one line. Split the points, grow trees locally without talking, then run Borůvka once more over the trees: the clusters never depend on the split (Theorem 3).

6 Details that decide correctness

> *The definitions are exact. Where can code still get them wrong?*

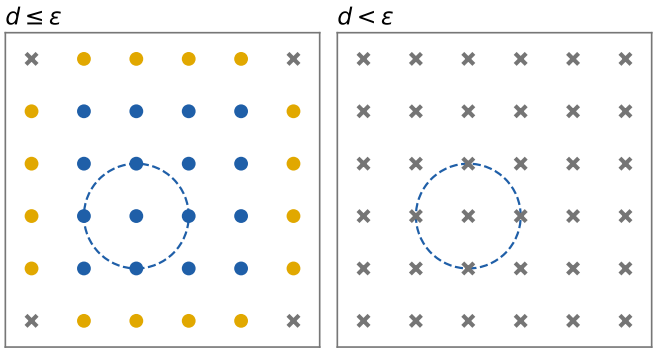
The paper's definitions leave a few decisions to the implementation. Each is fixed by the code and checked by a test; the first two were wrong in the original implementation.

Invariant 1 (the radius test). A distance equal to ε is within ε ($\leq$, as in Lemma 1 and in scikit-learn's DBSCAN). Distances and ε are compared in single precision, the precision of the stored graph: a double-precision ε is first rounded to the nearest `float32`.

1 Distances exactly at ε

Algorithm 1

```
⋮
Ensure: a cluster label for each of this rank's points, −1 for noise
C ← {i : distance in column M − 1 of row i ≤ ε}
R, cut edges ← local Borůvka over C
⋮
```



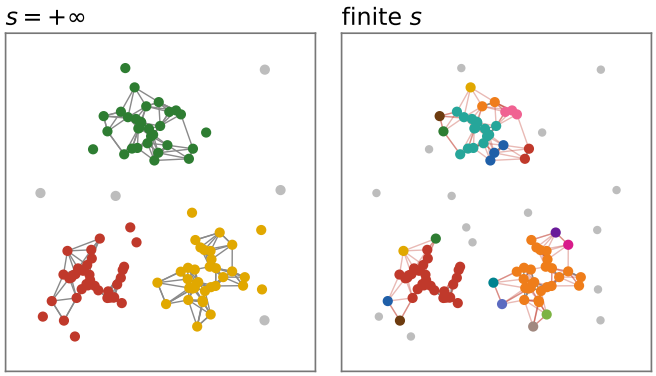
On a unit grid with $\varepsilon = 1$ and $M = 5$, each interior point has four neighbors at distance exactly ε. With $\leq$ the interior points are core ● and the rest border ● or noise. The strict $<$ of the original implementation finds no core point at all. Grids and rounded distances put many points exactly at ε.

Invariant 2 (no finite "no edge yet" weight). The weight that marks a subtree without a candidate edge is $+\infty$. Any finite sentinel s silently drops every edge of weight at least s, so the result would depend on the units of the data.

2 A finite "no edge" weight

Algorithm 2

```
⋮
R[i] ← i and L[i] ← 1 for every i ∈ C
repeat
    E_min[u] ← (none, +∞) for every subtree u
    parallel for i ∈ C
        while L[i] ≤ M − 1
            ⋮
```



With a finite s, edges of weight s or more (faint red) can never be picked, and clusters fall apart, here into 20 pieces. The original $s = 10^4$ did this to data whose neighbor distances reach 10^4, such as raw physical units; here s is the 60th percentile of the example's edge weights.

Invariant 3 (deterministic labels). Clusters are numbered $0, 1, \ldots$ in order of their lowest-indexed point, and noise is -1. The core's own numbering follows its roots, which depend on thread scheduling (Proposition 1); renumbering makes the labels a function of the partition alone, identical for any number of threads.

Invariant 4 (invariances). The clustering is unchanged by scaling all distances and ε by the same factor, by any strictly increasing transformation of distances and ε (the algorithm only compares them), by permuting the points (which permutes the labels), and by dropping graph columns beyond $M - 1$. The clusters of core points are unchanged by the number of threads and ranks (Theorem 3).

Invariant 5 (valid input). Each row lists neighbors other than the point, in nondecreasing order of finite, nonnegative distance, with $2 \leq M \leq k + 1$. The algorithm relies on all of it: border labelling stops at the first distance beyond ε and depends on sorted rows to stay inside its row, and an out-of-range index is read as another point's row. Before clustering, the library rejects any row with an index out of range, the point itself, or a distance that is negative, not finite or out of order, as well as an M outside that range.

In one line. Compare with $\leq$ in the graph's precision, never cap edge weights, and renumber the labels: then the result depends only on the graph, ε and M.

7 How we know it is right

> *A clustering has no expected value to compare with. How is it tested?*

Against the paper's statements, not against stored outputs. Six kinds of test, each run on the library, the MPI program and the C API:

Reference implementation
The definitions of Section 3 computed directly with sparse connected components (`tests/reference.py`), written from the paper, never from the C++ code. Core points, clusters and noise must match exactly, and each border point must join an allowed cluster.

Random directed graphs
Distances with no triangle inequality, rarely mutual edges, ties and zeros. Their picks form the cycles of Section 4.4 far more often than data does, and they found every defect in the cycle handling.

Theorems as tests
Classes equal scikit-learn's DBSCAN (Lemma 1); clusters refine DBSCAN's and coincide when the counts match (Theorem 1); a larger ε only adds core points and merges clusters.

Metamorphic tests
The invariances of Section 6, applied exactly: scaling by powers of two (exact in floating point, up to $2^{\pm 120}$), integer distances under affine and quadratic maps, permutations, truncated graphs.

Ranks and threads
The MPI program on 1 to 7 ranks, with points shuffled (most edges cut) or grouped (few cut) and with ranks that own nothing, compared with the reference (Theorem 3); repeated runs and thread counts must agree.

Sanitizers
Everything above under AddressSanitizer and UndefinedBehaviorSanitizer. Their builds change memory layout and timing, which exposed a data race that optimized builds hid.

The original implementation, which produced the paper's results, passed its own experiments. These tests found the following defects in it, all fixed in this implementation. None is in the method; each is a way code can break the method's assumptions, and each appears earlier in this companion where it arises.

Defect	Effect	Found by
Strict comparison $d < \varepsilon$	points exactly at ε were not core (grid with $\varepsilon = 1$: none instead of 16)	Lemma 3.1 test on a grid
Finite sentinel weight 10^4	edges of weight $\geq 10^4$ never merged (data scaled by 2^{16}: 109 clusters instead of 3)	scaling by powers of two
Power-of-two cycles in distributed pointer jumping	clusters split on some rank counts (one cluster became 6 on 7 ranks)	random graphs over rank counts
Grouping by overwriting representatives	endless loop in cycle joining (5 ranks)	random graphs over rank counts
Local cycle grouping read past its array	subtrees on cycles merged into point 0's cluster	AddressSanitizer
Loop variable shared by threads	core points labelled with another thread's root	sanitized builds, thread sweep
Every rank truncated the output file	empty labels file on oversubscribed ranks	MPI tests on CI runners

Table 1: Defects found in the original implementation.

The last two depended only on how threads or ranks were scheduled, which is why they appeared under sanitizers and on busy CI machines but rarely in ordinary runs:

1 A loop variable shared by threads

Algorithm 1

```
⋮
roots ← cut MST over the subtrees
R[i] ← root of R[i] for every i ∈ C
parallel for each non-core point i
⋮
```

	thread A	thread B	shared u
1	$u \leftarrow R[p]$		$R[p]$
2		$u \leftarrow R[q]$	$R[q]$
3	$R[p] \leftarrow \text{root}(u)$		$R[q]$

The loop that gives each core point its cluster used one variable u for all threads. Thread B could overwrite it between thread A's read and A's use, and point p received the cluster of B's point q. Declaring u inside the loop makes it private to each iteration.

2 One writer for the output file

In the command-line program, outside the algorithm: every rank opened, and so emptied, the labels file, but only rank 0 wrote it. A rank that reached the open after rank 0 had finished left an empty file. Only rank 0 may open it.

	rank 0	rank 2 (late)
1	open, truncate	*still computing*
2	write n labels	
3		open, truncate

A lesson for testing parallel graph code. All the cycle defects lived in paths that real data rarely reaches: exact kNN graphs of continuous data produce mostly mutual pairs (Section 4.4), and a few ranks rarely produce cycles whose length is a power of two. Randomized inputs that are valid but adversarial, swept over thread and rank counts and compared with an independent reference, reached them within seconds.

In one line. Test against the theorems with an independent reference, on adversarial random graphs, across thread and rank counts, under sanitizers.

8 Where the time goes

> *What does it cost, and what should be computed only once?*

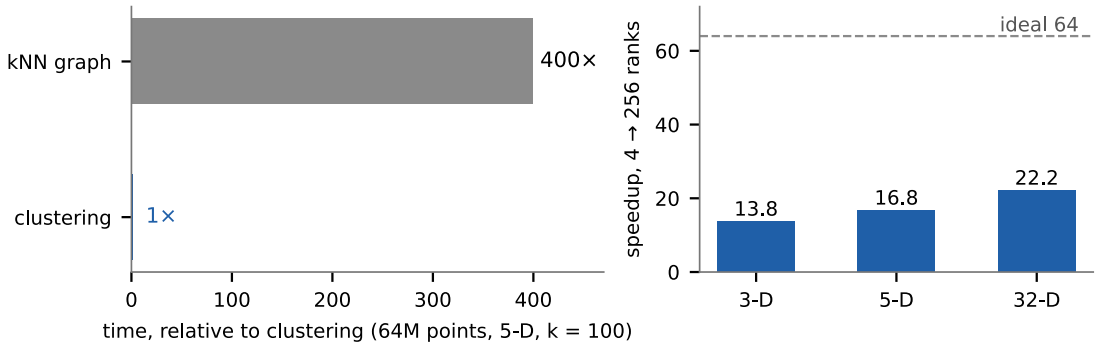


Figure 5: Left: building the kNN graph of 64 million 5-D points with $k = 100$ takes about 400 times longer than clustering it. Right: strong scaling of 64 million points from 4 to 256 ranks, against the ideal 64. Both from the paper's experiments on TACC's Frontera.

The graph dominates, so compute it once and reuse it across ε and M (Section 3). Approximate graphs are acceptable: kNN-DBSCAN is defined on whatever graph it is given, and in the paper's experiments lowering the graph's accuracy to 75 % did not change the clustering quality. Scaling is limited by cut edges: with more

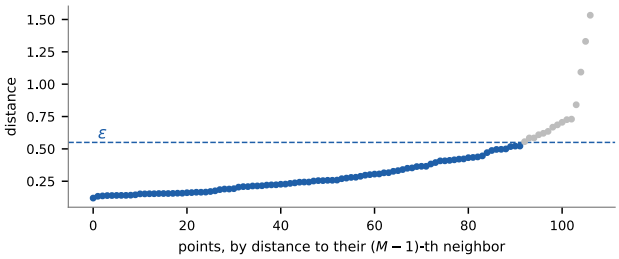
ranks, local work shrinks but the fraction of cut edges, and the communication per Borůvka round, grows, and it grows with dimension too. At scale, the paper clusters one billion 3-D points in under a second on 28K cores, 65 billion 20-D points in under 40 s on 114,688 cores, and runs up to 37× faster than PDBSCAN [5] on 4 million 20-D points.

On a single node, the clustering stage of the Python package takes about 0.9 s for a 4-million-point graph with 19 neighbors per point on one thread and 0.2–0.26 s on eight, with no further gain at 16 on a 16-thread laptop. Checking the input graph and renumbering labels add 5–10 %. Logging costs nothing when disabled.

9 Using it

Which ε and M?

Choose M first, as for DBSCAN: a larger M smooths the density estimate. Then sort the points by the distance to their $(M-1)$-th neighbor, the number that decides whether a point is core (Lemma 1). Dense points form the flat part of the curve and sparse points its steep end; an ε at the elbow [6] makes the flat part core ●.



Compute a graph with $k \geq M-1$, larger if several M will be tried, and cluster it for each candidate:

```
from knndbscan import KNNDBSCAN, knn_dbscan, knn_dbscan_from_graph

labels = knn_dbscan(X, eps=0.05, min_samples=15, n_jobs=-1)            # exact graph
labels = knn_dbscan_from_graph(neighbors, distances, eps=0.05, min_samples=15)
model = KNNDBSCAN(eps=0.05, min_samples=15).fit(X)   # labels_, core_sample_indices_
```

For graphs that do not fit on one node, the MPI program reads a graph file and runs on any number of ranks:

```
mpiexec -n 64 knndbscan -n N -e eps -m M -k K -t threads -i graph.txt -o labels.txt
```

References

[1] Y. Chen, W. Ruys, and G. Biros, “KNN-DBSCAN: a DBSCAN in high dimensions,” 2024. [Online]. Available: https://arxiv.org/abs/2009.04552

[2] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[3] B. Xiao and G. Biros, “Parallel algorithms for nearest neighbor search problems in high dimensions,” *SIAM Journal on Scientific Computing*, vol. 38, no. 5, 2016.

[4] C. D. Yu, S. Reiz, and G. Biros, “Distributed-memory hierarchical compression of dense SPD matrices,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC18)*, 2018.

[5] M. M. A. Patwary, D. Palsetia, A. Agrawal, W.-keng Liao, F. Manne, and A. Choudhary, “A new scalable parallel DBSCAN algorithm using the disjoint-set data structure,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC12)*, 2012.

[6] E. Schubert, J. Sander, M. Ester, H.-P. Kriegel, and X. Xu, “DBSCAN revisited, revisited: why and how you should (still) use DBSCAN,” *ACM Transactions on Database Systems*, vol. 42, no. 3, 2017.